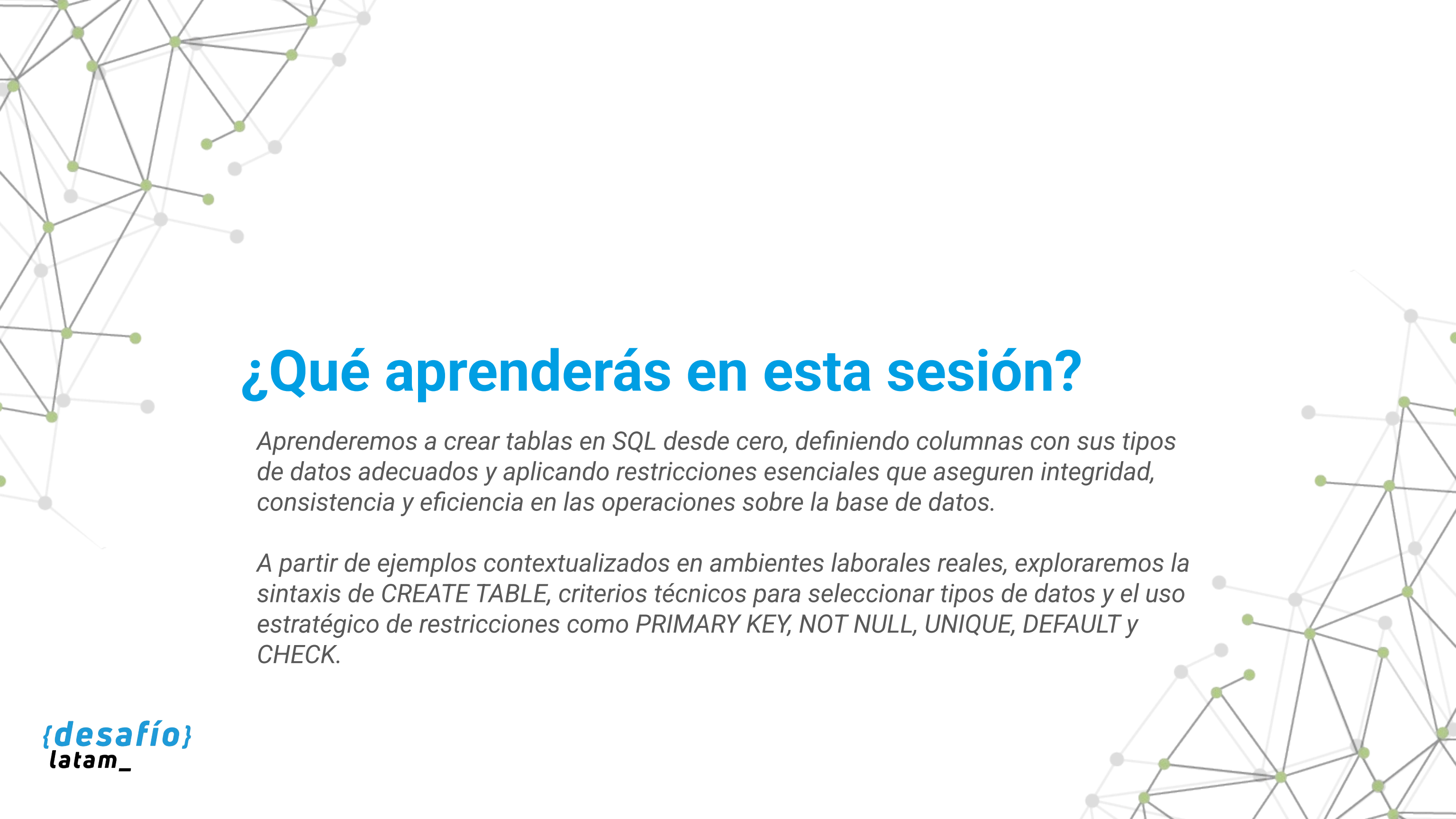




Manipulación y Gestión de Datos

Clase 1 - Creación y manipulación de tablas (Parte I)



¿Qué aprenderás en esta sesión?

Aprenderemos a crear tablas en SQL desde cero, definiendo columnas con sus tipos de datos adecuados y aplicando restricciones esenciales que aseguren integridad, consistencia y eficiencia en las operaciones sobre la base de datos.

A partir de ejemplos contextualizados en ambientes laborales reales, exploraremos la sintaxis de `CREATE TABLE`, criterios técnicos para seleccionar tipos de datos y el uso estratégico de restricciones como `PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `DEFAULT` y `CHECK`.

Desafío inicial

Estás a cargo del modelado de una nueva base de datos para el área de gestión de capacitaciones de una empresa de telecomunicaciones. Se requiere almacenar información sobre las personas participantes, los cursos ofrecidos y sus respectivas inscripciones. El equipo de tecnología necesita una estructura de base de datos que garantice consistencia, evite redundancias y prevenga errores de ingreso.

¿Cómo diseñar tablas que reflejen fielmente la realidad del proceso de capacitación, seleccionando tipos de datos adecuados y restricciones que aseguren integridad referencial y validez lógica desde el inicio?

Para resolverlo debes ser capaz de modelar una tabla con sus columnas, seleccionar tipos de datos apropiados según el tipo de información, y aplicar restricciones que impidan errores comunes como duplicación de registros, datos en blanco o valores fuera de rango.

Creación de una tabla: estructura base de datos relacional

¿Por qué importa diseñar correctamente una tabla?

En el ciclo de vida de una base de datos, la creación de tablas es una actividad crítica. Si se definen mal, arrastrarán errores lógicos, dificultades de mantenimiento, duplicidades de información y restricciones poco útiles. En ambientes productivos, una tabla mal construida puede afectar la trazabilidad, los informes gerenciales y la toma de decisiones basada en datos.

Estructura mínima y sintaxis de CREATE TABLE

```
CREATE TABLE nombre_tabla (  
    nombre_columna tipo_dato [restricciones],  
    ...  
);
```

Componentes esenciales

Elemento	Descripción
Nombre de tabla	Identificador único del objeto tabla
Columnas	Campos que almacenan atributos de la entidad
Tipos de datos	Definen el formato y las operaciones permitidas
Restricciones	Controlan validez, unicidad, obligatoriedad, integridad

Tabla 1. Componentes esenciales de una tabla

Fuente: Desafío Latam

Ejemplo aplicado real

```
CREATE TABLE empleados (  
    id_empleado SERIAL PRIMARY KEY,  
    rut VARCHAR(12) UNIQUE NOT NULL,  
    nombre_completo VARCHAR(100) NOT NULL,  
    fecha_ingreso DATE DEFAULT CURRENT_DATE,  
    salario_base DECIMAL(10,2) CHECK (salario_base > 0)  
);
```

Este diseño corresponde a un sistema de gestión de RRHH en una empresa logística. El RUT no debe repetirse; el salario no puede ser negativo; y la fecha de ingreso debe poder ser completada automáticamente al registrar un nuevo empleado.

Buenas prácticas recomendadas



Declarar la clave primaria explícitamente.



Usar SERIAL o IDENTITY en campos numéricos incrementales.



Aplicar NOT NULL a campos obligatorios.



Utilizar nombres descriptivos y consistentes (snake_case).



Evitar tipos de datos genéricos como TEXT si no son necesarios.

Errores comunes en la creación de tablas

No declarar una clave primaria, dificultando la relación con otras tablas.

Usar VARCHAR(255) por defecto, sin analizar el tamaño real necesario.

Omitir restricciones básicas como CHECK para validar rangos de valores.

Repetir columnas que podrían ser derivadas (por ejemplo, edad en vez de fecha de nacimiento).

Estas malas prácticas afectan la normalización, la validación de datos y el rendimiento en operaciones de consulta o actualización.

Aplicación productiva y control de calidad de estructuras

Una tabla correctamente diseñada permite:



Control de duplicación

Controlar duplicación de información.



Auditorías

Facilitar auditorías internas y cumplimiento normativo.



Integración analítica

Integrarse con herramientas analíticas que esperan estructuras limpias.



Independencia

Reducir la dependencia de reglas de validación en el frontend.

Métrica clave (indicativa)

% de tablas con claves primarias definidas / total de tablas del sistema. Esta métrica puede utilizarse como un indicador de madurez estructural en una base de datos organizacional.

Dominar la creación de tablas es el primer paso hacia un modelado profesional de datos. Pero una tabla bien definida no solo depende de la sintaxis general: también requiere definir correctamente el tipo de dato de cada columna, considerando precisión, formato, almacenamiento y validación.

A continuación, abordaremos los criterios técnicos para definir tipos de datos adecuados según el tipo de información a registrar.



Tipos de datos numéricos

Enteros

INT, SMALLINT, BIGINT: para enteros de diferentes rangos.

Precisión exacta

DECIMAL(p, s), NUMERIC(p, s): para valores con precisión exacta, como montos monetarios.

Punto flotante

FLOAT, REAL, DOUBLE PRECISION: para cálculos científicos o donde la precisión decimal no sea crítica.

Ejemplo aplicado

```
salario_base DECIMAL(10,2) CHECK (salario_base >= 0)
```

En sistemas de nómina, se requiere exactitud para cálculos financieros y reportes tributarios.

Tipos de datos de texto

CHAR(n)

Tamaño fijo. Útil para códigos estándar como RUT, códigos postales.

VARCHAR(n)

Longitud variable. Ideal para nombres, direcciones o emails.

TEXT

Uso excepcional para observaciones largas (no indexadas).

Buenas prácticas

- Evitar TEXT si no es estrictamente necesario.
- Limitar VARCHAR a longitudes reales para optimizar memoria.

Ejemplo:

```
correo_electronico VARCHAR(80) NOT NULL
```

Tipos de datos de fecha y hora

DATE

Fechas sin hora (ideal para cumpleaños, fechas de contratación).

TIME

Hora sin fecha (horarios laborales).

TIMESTAMP

Fecha y hora combinadas.

Ejemplo:

```
fecha_inscripcion DATE NOT NULL
```

Aplicación: Control preciso del ingreso de estudiantes o emisión de certificados.

Criterios técnicos para elegir un tipo de dato

Requisito de negocio	Tipo de dato sugerido	Justificación
Montos financieros exactos	DECIMAL(10,2)	Precisión garantizada para cálculos tributarios y contables
ID incremental	SERIAL	Generación automática, sin colisiones
Nombre de personas	VARCHAR(100)	Longitud razonable para nombres en Latinoamérica
Fechas de inscripción	DATE	Permite operaciones de comparación y ordenamiento temporal
Campos booleanos	BOOLEAN o CHAR(1)	Según compatibilidad del motor

Tabla 2. Criterios técnicos para elegir un tipo de dato

Fuente: Desafío Latam

Errores comunes al definir tipos de datos

Usar FLOAT para dinero → provoca errores de redondeo.

Definir VARCHAR(255) por defecto en todos los campos → desperdicio de espacio.

No establecer DEFAULT cuando se espera un valor por omisión.

Almacenar fechas como VARCHAR → impide ordenamientos correctos y validaciones.

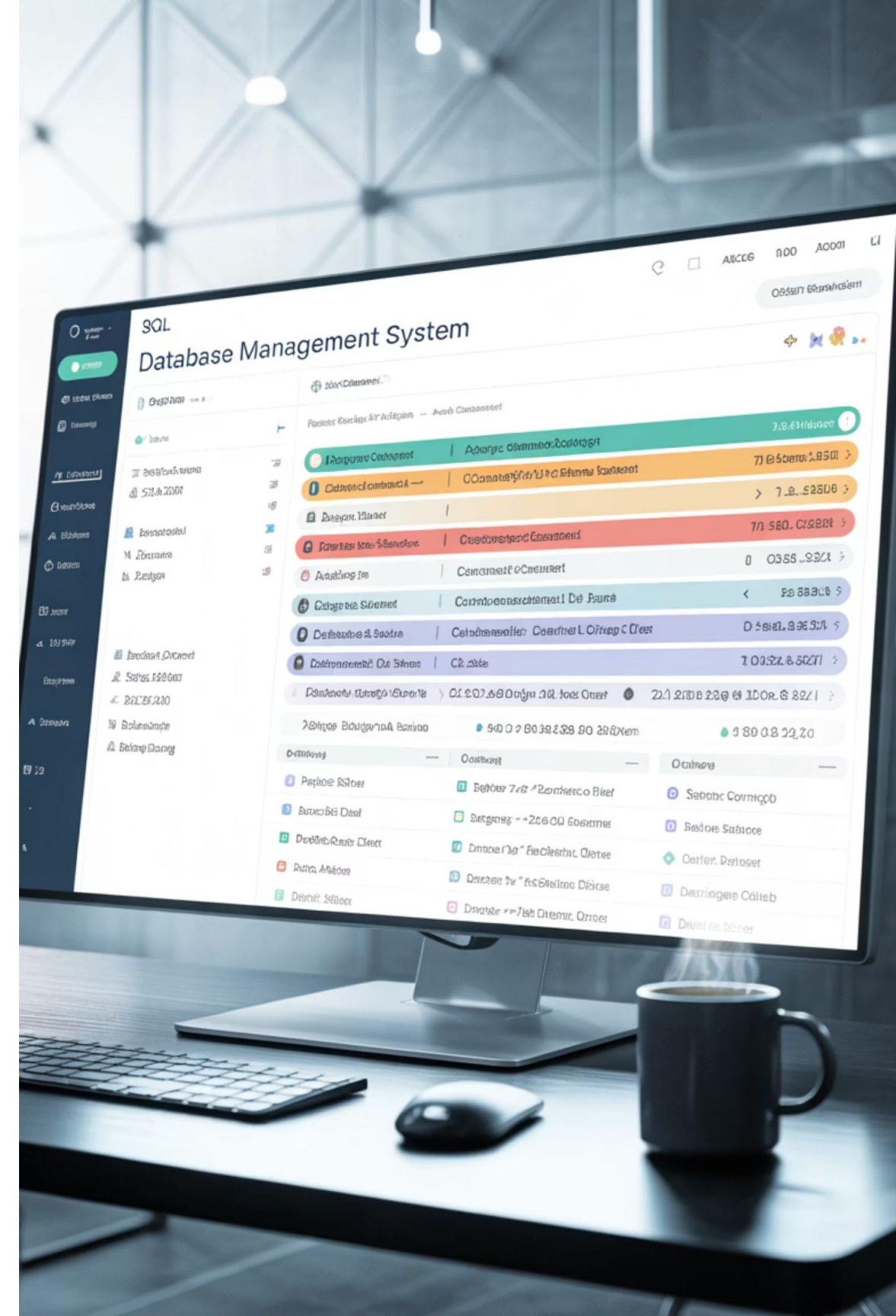
Bases mal optimizadas, datos inconsistentes, pérdida de rendimiento y necesidad de validaciones externas.

La correcta elección del tipo de dato permite a las bases de datos validar automáticamente la entrada, optimizar recursos y facilitar integraciones posteriores. A partir de esta base técnica, pasaremos ahora a revisar las restricciones y especificaciones que pueden aplicarse a cada columna para asegurar la integridad lógica de los registros y prevenir errores estructurales desde el origen del dato.

Restricciones y especificaciones en columnas

¿Cómo pueden las restricciones mejorar la calidad de los datos desde su ingreso?

En bases de datos relacionales, las restricciones en las columnas permiten garantizar que los datos ingresados respeten reglas lógicas, de unicidad, obligatoriedad y coherencia. En entornos productivos, esto es clave para prevenir errores humanos, inconsistencias y duplicaciones, especialmente cuando múltiples sistemas y personas interactúan con los datos.



Tipos de restricciones más comunes

Restricción	Descripción
PRIMARY KEY	Identifica de forma única cada fila. No permite valores nulos ni duplicados.
NOT NULL	Impide que una columna tenga valores nulos.
UNIQUE	Garantiza que todos los valores en la columna sean distintos.
DEFAULT	Define un valor por defecto si no se especifica otro.
CHECK	Establece condiciones lógicas que deben cumplirse para aceptar el dato.
FOREIGN KEY	Relaciona una columna con una clave primaria en otra tabla.

Tabla 3. Tipos de restricciones más comunes

Fuente: Desafío Latam

Ejemplos aplicados en contexto productivo

Caso 1: Control de registros de asistencia a capacitaciones

```
CREATE TABLE asistencia (  
    id SERIAL PRIMARY KEY,  
    id_participante INT NOT NULL,  
    id_sesion INT NOT NULL,  
    estado CHAR(1) DEFAULT 'P' CHECK (estado IN ('P','A','J'))  
);
```

Se usa CHECK para asegurar que el campo estado solo acepte valores válidos: Presente (P), Ausente (A) o Justificado (J).

Caso 2: Evitar duplicados en inscripciones

```
CREATE TABLE inscripciones (  
    id_participante INT NOT NULL,  
    id_curso INT NOT NULL,  
    fecha_inscripcion DATE DEFAULT CURRENT_DATE,  
    PRIMARY KEY (id_participante, id_curso)  
);
```

Se define una clave primaria compuesta para evitar que una misma persona se inscriba dos veces en el mismo curso.

Buenas prácticas en el uso de restricciones



Usar NOT NULL siempre que el campo sea obligatorio por lógica de negocio.



Aplicar CHECK para validar rangos, formatos o condiciones específicas.



Definir valores por defecto que reflejen el flujo natural de datos.



Documentar las restricciones con comentarios (COMMENT ON COLUMN) cuando sea posible.



Probar las restricciones con datos reales o simulados antes de pasar a producción.

Las restricciones implementadas a nivel de base de datos actúan como una segunda capa de validación, complementaria a la lógica de aplicación.

Errores comunes al definir restricciones

Declarar UNIQUE en columnas que pueden tener valores nulos repetidos (p. ej., segundo apellido).

Omitir CHECK cuando se conocen rangos válidos (p. ej., edades entre 18 y 65).

Confiar exclusivamente en la aplicación para validar datos, sin respaldo en la BD.

Crear claves primarias sin considerar si los valores serán realmente únicos.

Duplicación de datos, errores lógicos, pérdida de confiabilidad, necesidad de refactorización costosa.

El uso correcto de restricciones no solo fortalece la estructura de las tablas, sino que también reduce significativamente los errores de ingreso y mejora la calidad de los reportes. En ambientes colaborativos y de alto volumen, su uso es una garantía de consistencia y seguridad.

Actividad guiada:

**Modelado de tablas para
gestión de capacitaciones**



Objetivo de la actividad

Aplicar los conocimientos adquiridos sobre creación de tablas, definición de tipos de datos y aplicación de restricciones, construyendo una estructura de base de datos coherente con un escenario productivo.

Situación problema

Una empresa de formación técnica desea crear un sistema para gestionar sus procesos de inscripción.

La base de datos debe contemplar las siguientes entidades: personas participantes, cursos e inscripciones. Es necesario crear las tablas que representan estas entidades, cuidando la correcta selección de tipos de datos y la implementación de restricciones que aseguren la integridad y coherencia de los datos.



Script SQL base para la actividad.

```
-- Tabla participantes
CREATE TABLE participantes (
    id_participante SERIAL PRIMARY KEY,
    rut VARCHAR(12) UNIQUE NOT NULL,
    nombre_completo VARCHAR(100) NOT NULL,
    correo_electronico VARCHAR(80) NOT NULL,
    fecha_nacimiento DATE CHECK (fecha_nacimiento <= CURRENT_DATE)
);

-- Tabla cursos
CREATE TABLE cursos (
    id_curso SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    duracion_horas INT CHECK (duracion_horas > 0),
    modalidad VARCHAR(20) DEFAULT 'Online',
    fecha_inicio DATE
);

-- Tabla inscripciones
CREATE TABLE inscripciones (
    id_participante INT NOT NULL,
    id_curso INT NOT NULL,
    fecha_inscripcion DATE DEFAULT CURRENT_DATE,
    PRIMARY KEY (id_participante, id_curso),
    FOREIGN KEY (id_participante) REFERENCES participantes(id_participante),
    FOREIGN KEY (id_curso) REFERENCES cursos(id_curso)
);
```



Instrucciones



Diseño

Diseña y crea las tablas participantes, cursos e inscripciones según el escenario descrito.



Restricciones

Aplica restricciones como PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT y CHECK donde corresponda.



Tipos de datos

Define los tipos de datos más adecuados para cada campo.



Relaciones

Asegúrate de que la tabla de inscripciones relacione correctamente a los participantes con los cursos.

Materiales necesarios

- Acceso a un entorno SQL (p. ej., DB Fiddle, pgAdmin, DBeaver o entorno local).
- Planificador o pizarra digital para discutir el modelo.

Actividad práctica autónoma:

Diseño de tabla de proveedores



Objetivo de la actividad

Aplicar el diseño de una tabla relacional, seleccionando tipos de datos y restricciones adecuadas para un nuevo escenario productivo vinculado al control de proveedores.

Situación problema

Trabajas en el área de adquisiciones de una empresa que desea mejorar su registro de proveedores.

Se busca definir una estructura de tabla que almacene datos esenciales, como identificación, tipo de proveedor, condiciones de pago y estado de habilitación, garantizando consistencia y validez desde la base de datos.



Instrucciones para desarrollar la actividad

1

Diseña una tabla llamada proveedores con al menos 6 campos relevantes.

2

Define tipos de datos adecuados para cada campo, considerando variedad entre campos numéricos, de texto y lógicos.

3

Aplica al menos cuatro restricciones: PRIMARY KEY, UNIQUE, NOT NULL, DEFAULT, CHECK.

4

Documenta tus decisiones en la siguiente tabla:

Campo (columna)	Tipo de dato	Restricción aplicada	Justificación breve

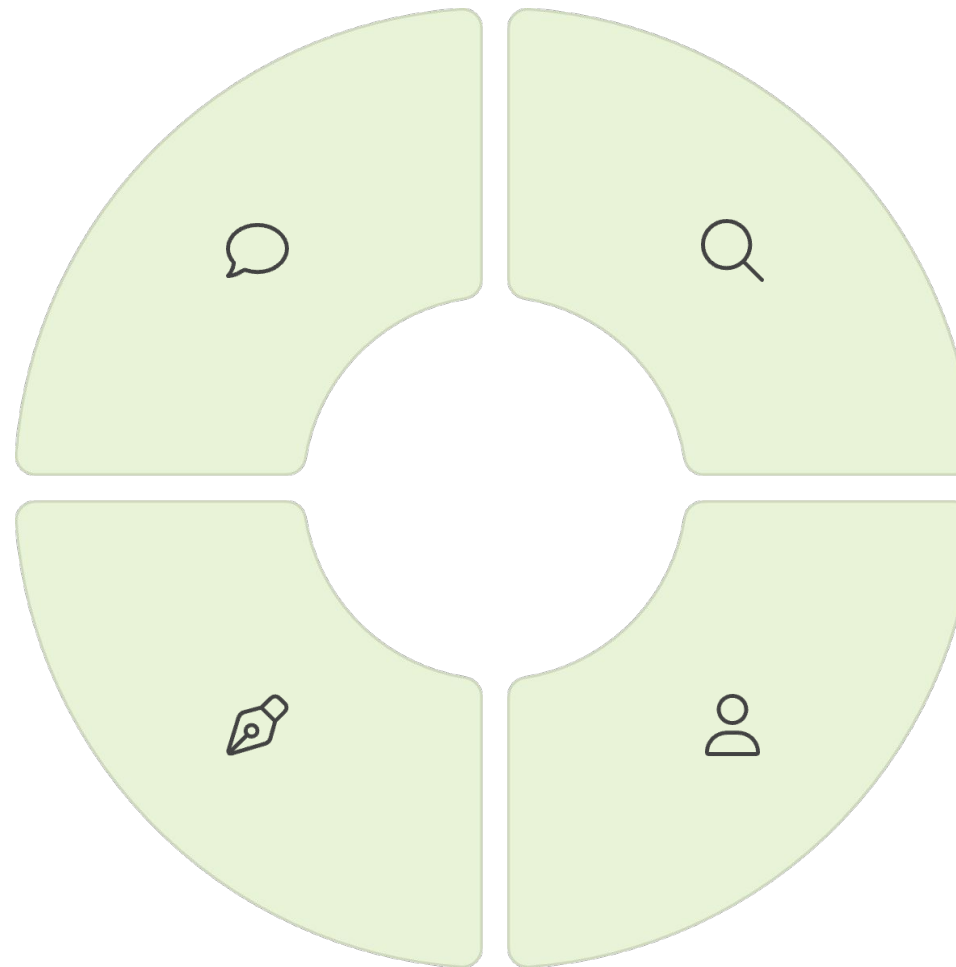
Resumen

CREATE TABLE

Conocimos la sintaxis básica de CREATE TABLE y cómo definir correctamente una tabla.

Buenas prácticas

Identificamos buenas prácticas y errores comunes en el diseño de estructuras relacionales.



Tipos de datos

Aprendimos a seleccionar tipos de datos apropiados según el tipo de información a almacenar.

Restricciones

Aplicamos restricciones como PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT y CHECK para garantizar integridad y coherencia.



Próxima sesión...

En la siguiente sesión nos enfocaremos en la segunda parte de este proceso: la inserción manual de valores en las tablas.

Aprenderemos a utilizar la instrucción `INSERT INTO`, comprenderemos su estructura, sintaxis y variantes, y exploraremos estrategias para ingresar datos de forma segura, coherente y trazable.

Además, analizaremos cómo validar si los datos insertados respetan las restricciones establecidas.

Preguntas de cierre

¿Qué diferencia hay entre NOT NULL y CHECK?

¿Qué implicancia tiene elegir VARCHAR(100) frente a TEXT en una columna?

¿Qué tipo de errores puedes evitar al usar DEFAULT en columnas estratégicas?

¿Cuándo conviene usar una clave primaria compuesta?

¿Qué consecuencias puede tener definir una columna como UNIQUE si permite valores nulos?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2019). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
- Oppel, A. J. (2010). Databases Demystified (2nd ed.). McGraw-Hill.
- PostgreSQL Documentation. (2024). Data Types and Table Constraints. <https://www.postgresql.org/docs/current/>
- Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.